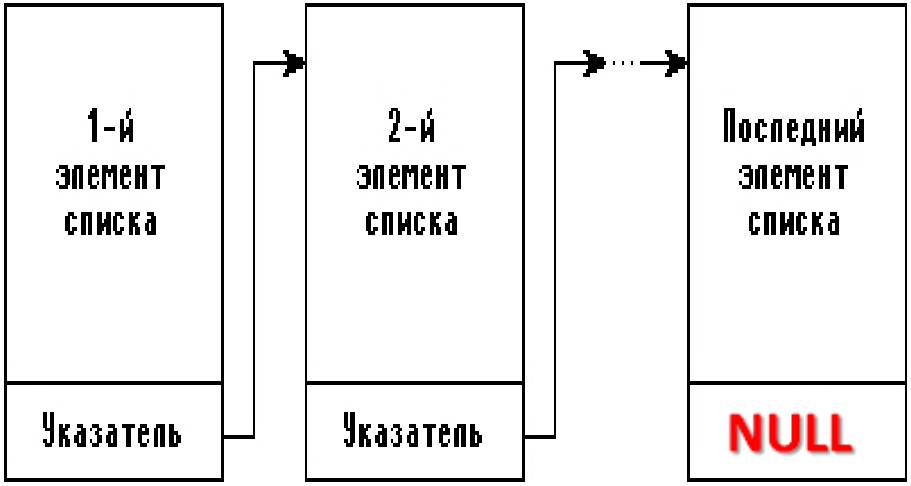
Динамические структуры

Данные динамической структуры
Несвязанные динам. структуры
Связанные динам. структуры
.....
Классифицируются аналогично данным статической структуры
.....
Линейной структуры
Кольцевой структуры
Разветвленной структуры
Односвязные
Очередь
Стек
Дек
Список
Многосвязные
Многосвязный линейный список
Односвязный кольцевой список
Многосвязный кольцевой список
Деревья
Графы
Двоичные (бинарные)
Разветвленные

Односвязный линейный список

```
struct list
{
  int field; // поле данных
  struct list *ptr; // указатель на
// следующий элемент
};
```



Линейная структура

Основные действия, производимые над элементами

- Инициализация списка
- Добавление узла в список
- Удаление узла из списка
- Удаление корня списка
- Вывод элементов списка
- Взаимообмен двух узлов списка

Инициализация списка

```
struct list * init(int a) // a- значение первого узла
{
  struct list *lst;
  // выделение памяти под корень списка
  lst = (struct list*)malloc(sizeof(struct list));
  lst->field = a;
  lst->ptr = NULL; // это последний узел списка
  return(lst);
}
```

Добавление элемента

```
struct list * addelem(list *lst, int number)
{
  struct list *temp, *p;
  temp = (struct list*)malloc(sizeof(list));
  p = lst->ptr; // сохранение указателя на следующий узел
  lst->ptr = temp; // предыдущий узел указывает на создаваемый
  temp->field = number; // сохранение поля данных добавляемого узла
  temp->ptr = p; // созданный узел указывает на следующий элемент
  return(temp);
}
```

Удаление элемента

```
struct list * deletelem(list *lst, list *root)
{
  struct list *temp;
  temp = root;
  while (temp->ptr != lst) // просматриваем список начиная с корня
  { // пока не найдем узел, предшествующий lst
    temp = temp->ptr;
  }
  temp->ptr = lst->ptr; // переставляем указатель
  free(lst); // освобождаем память удаляемого узла
  return(temp);
}
```

Удаление корня

```
struct list * deletehead(list *root)
{
  struct list *temp;
  temp = root->ptr;
  free(root); // освобождение памяти текущего корня
  return(temp); // новый корень списка
}
```

Вывод элементов списка

```
void listprint(list *lst)
{
  struct list *p;
  p = lst;
  do {
    printf("%d ", p->field); // вывод значения элемента p
    p = p->ptr; // переход к следующему узлу
  } while (p != NULL);
}
```

Односвязный циклический список

```
struct list * init(int a) // a- значение первого узла
{
  struct list *lst;
  // выделение памяти под корень списка
  lst = (struct list*)malloc(sizeof(struct list));
  lst->field = a;
  lst->ptr = lst; // указатель на сам корневой узел
  return(lst);
}
```

Односвязный циклический список

```
void listprint(list *lst)
{
  struct list *p;
  p = lst;
  do {
    printf("%d ", p->field); // вывод значения узла p
    p = p->ptr; // переход к следующему узлу
  } while (p != lst); // условие окончания обхода
}
```

Двусвязный линейный список

```
struct list
{
  int field; // поле данных
  struct list *next; // указатель на следующий элемент
  struct list *prev; // указатель на предыдущий элемент
};
```

Двусвязный линейный список

```
struct list * init(int a)  // a- значение первого узла
{
  struct list *lst;
  // выделение памяти под корень списка
  lst = (struct list*)malloc(sizeof(struct list));
  lst->field = a;
  lst->next = NULL; // указатель на следующий узел
  lst->prev = NULL; // указатель на предыдущий узел
  return(lst);
}
```

Двусвязный линейный список

Двусвязный линейный список

```
struct list * addelem(list *lst, int number)
{
  struct list *temp, *p;
  temp = (struct list*)malloc(sizeof(list));
  p = lst->next; // сохранение указателя на следующий узел
  lst->next = temp; // предыдущий узел указывает на создаваемый
  temp->field = number; // сохранение поля данных добавляемого узла
  temp->next = p; // созданный узел указывает на следующий узел
  temp->prev = lst; // созданный узел указывает на предыдущий узел
  if (p != NULL)
    p->prev = temp;
  return(temp);
}
```

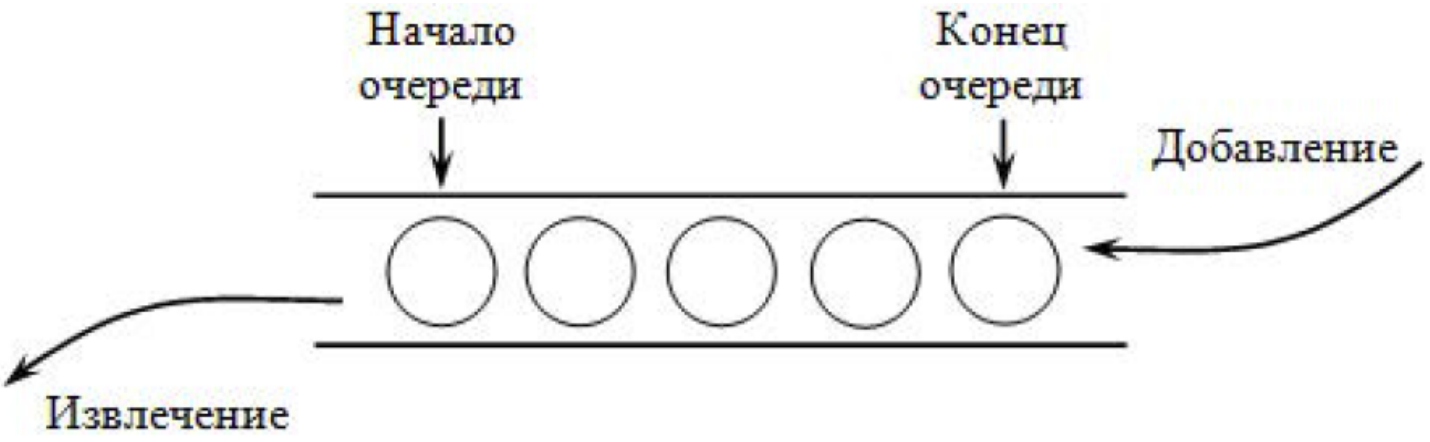
Двусвязный циклический список

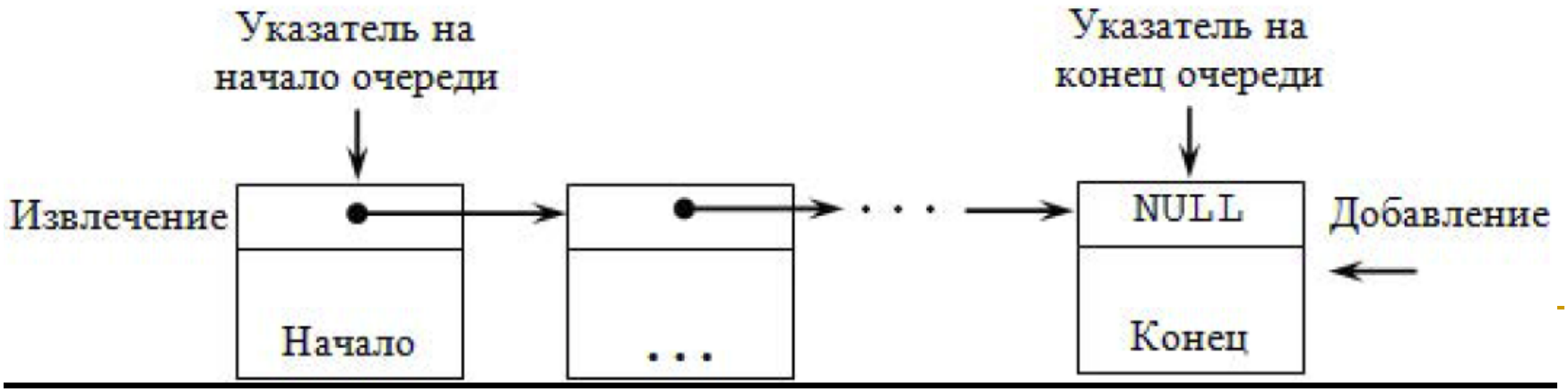
Двусвязный циклический список

```
struct list * init(int a)  // a- значение первого узла
{
  struct list *lst;
  // выделение памяти под корень списка
  lst = (struct list*)malloc(sizeof(struct list));
  lst->field = a;
  lst->next = lst; // указатель на следующий узел
  lst->prev = lst; // указатель на предыдущий узел
  return(lst);
}
```

Очередь

FIFO - First In First Out





Очередь

```
struct list {
  int field;
  struct list *ptr;
};
struct queue {
  struct list *frnt, *rear;
};
```

Основные действия над очередью

Создание очереди

```
void init(struct queue *q) {
  q->frnt = 0;
  q->rear = 0;
}
```

Проверка очереди на пустоту

```
int isempty(struct queue *q) {
  if(q->frnt==0)
    return 1;
  else
    return 0;
}
```

Добавление в очередь

```
void insert(struct queue *q, int x) {
  if((q->rear==0) && (q->frnt==0)) {
    q->rear = init(x);
    q->frnt = q->rear;
  } else
    q->rear = addelem(q->rear, x);
}
```

Вывод элементов очереди

```
void print(struct queue *q) {
  struct list *h;
  if(isempty(q)==1) {
    printf("Очередь пуста!\n");
    return;
  }
  for(h = q->frnt; h!= NULL; h=h->ptr)
    printf("%d ",h->field);
  return;
}
```

Удаление из очереди

```
int remove(struct queue *q) {
  struct list *temp;
  int x;
  if(isempty(q)==1) {
    printf("Очередь пуста!\n");
    return 0;
  }
  x = q->frnt->field;
  temp = q->frnt;
  q->frnt = q->frnt->ptr;
  free(temp);
  return x;
}
```

Стек

Стеком называется упорядоченный набор элементов, в котором размещение новых и удаление существующих происходит с одного конца, называемого ***вершиной***.

Дисциплина обслуживания — это совокупность правил (упорядочение и алгоритм) обслуживания элементов динамической структуры данных.

В зависимости от дисциплины обслуживания различают те или иные структуры динамических данных.

Принцип работы стека сравнивают со стопкой листов бумаги: чтобы взять второй сверху, нужно снять верхний.

В стеке реализуется дисциплина обслуживания LIFO:

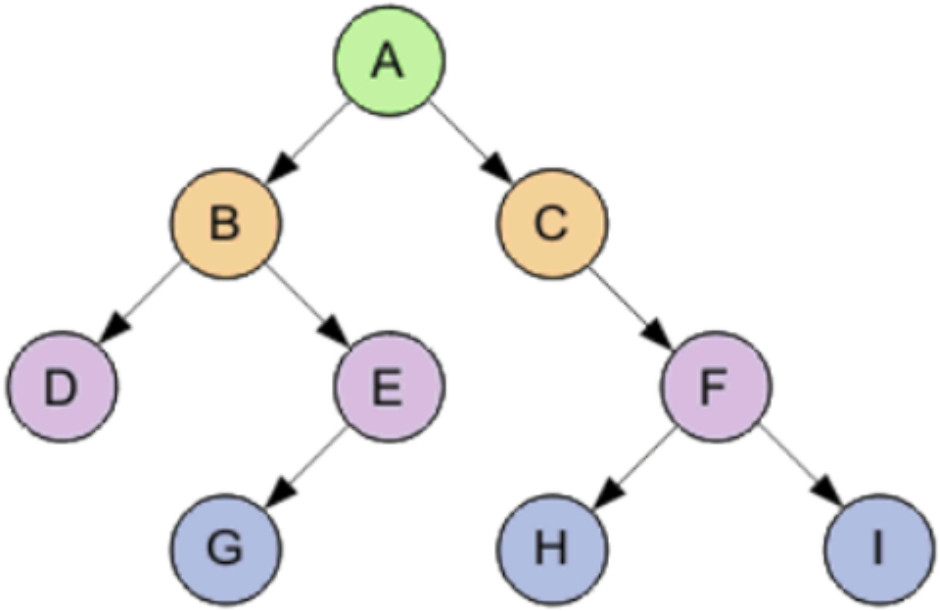
- ***LAST***— последний
- ***INPUT*** — вошел
- ***FIRST*** — первый
- ***OUTPUT*** — вышел

Дерево

Дерево – структура данных, представляющая собой древовидную структуру в виде набора связанных узлов.

Бинарное дерево — это конечное множество элементов, которое либо пусто, либо содержит элемент (***корень***), связанный с двумя различными бинарными деревьями, называемыми ***левым и правым поддеревьями***. Каждый элемент бинарного дерева называется ***узлом***. Связи между узлами дерева называются его ***ветвями***.

Способ представления бинарного дерева:



- A — корень дерева
- B — корень левого поддерева
- C — корень правого поддерева

Дерево

Корень дерева расположен на уровне с минимальным значением.

Узел D, который находится непосредственно под узлом B, называется ***потомком*** B.

Если D находится на уровне i, то B – на уровне i-1. Узел B называется ***предком*** D.

Максимальный уровень какого-либо элемента дерева называется его ***глубиной*** или ***высотой***.

Если элемент не имеет потомков, он называется ***листом*** или ***терминальным узлом*** дерева.

Остальные элементы – ***внутренние узлы*** (узлы ветвления).

Число потомков внутреннего узла называется его ***степенью***. Максимальная степень всех узлов есть степень дерева.

Реализация дерева

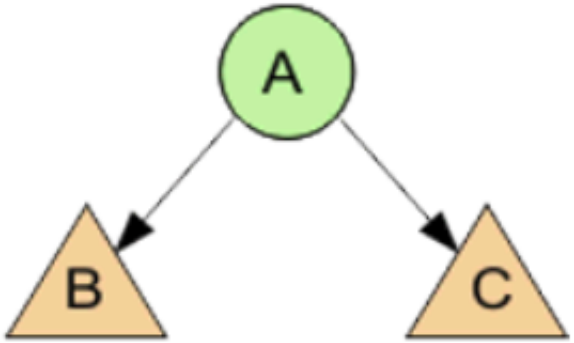
Узел дерева можно описать как структуру:

```
struct tnode {
  int field;            // поле данных
  struct tnode *left;   // левый потомок
  struct tnode *right;  // правый потомок
};
```

Дерево

Способы обхода дерева

Пусть имеем дерево, где A — корень, B и C — левое и правое поддеревья.



Существует три способа обхода дерева:

- Обход дерева сверху вниз (в прямом порядке): A, B, C — префиксная форма.
- Обход дерева в симметричном порядке (слева направо): B, A, C — инфиксная форма.
- Обход дерева в обратном порядке (снизу вверх): B, C, A — постфиксная форма.

Дерево

обход дерева в префиксной форме

```
void treeprint(tnode *tree) {
  if (tree!=NULL) { //Пока не встретится пустой узел
    printf(“%d \n”,tree->field); //Отображаем корень дерева
    treeprint(tree->left); //Рекурсивная функция для левого
поддерева
    treeprint(tree->right); //Рекурсивная функция для правого
поддерева
  }
}
```

Дерево

Обход дерева в инфиксной форме будет иметь вид

```
void treeprint(tnode *tree) {
  if (tree!=NULL) { //Пока не встретится пустой узел
    treeprint(tree->left); //Рекурсивная функция для левого поддерева
   printf("%d \n",tree->field);  /Отображаем корень дерева
    treeprint(tree->right); //Рекурсивная функция для правого поддерева
  }
}
```

Обход дерева в постфиксной форме будет иметь вид

```
void treeprint(tnode *tree) {
  if (tree!=NULL) { //Пока не встретится пустой узел
    treeprint(tree->left); //Рекурсивная функция для левого поддерева
    treeprint(tree->right); //Рекурсивная функция для правого поддерева
  printf("%d \n",tree->field);  //Отображаем корень дерева
  }
}
```